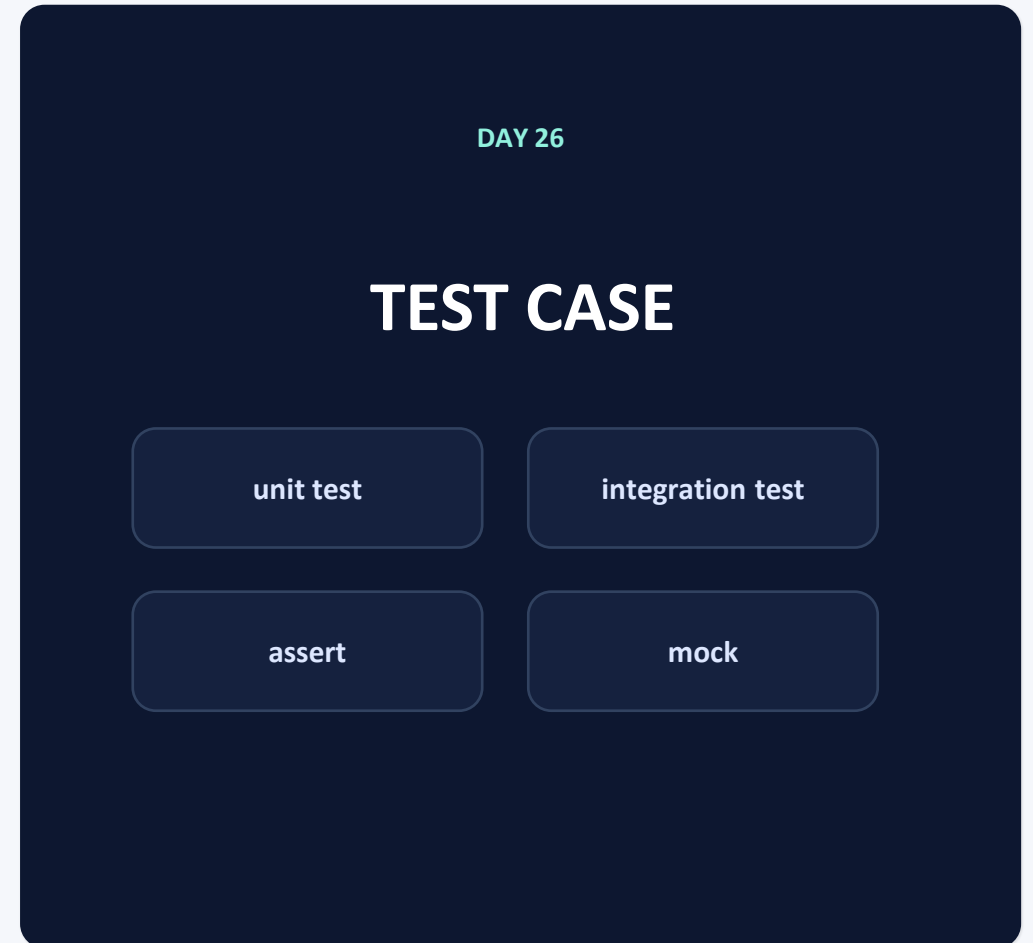


Тестирование

Писать простые unit-тесты, понимать assert, fixtures/mocks на базовом уровне и различать unit, integration и regression testing.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

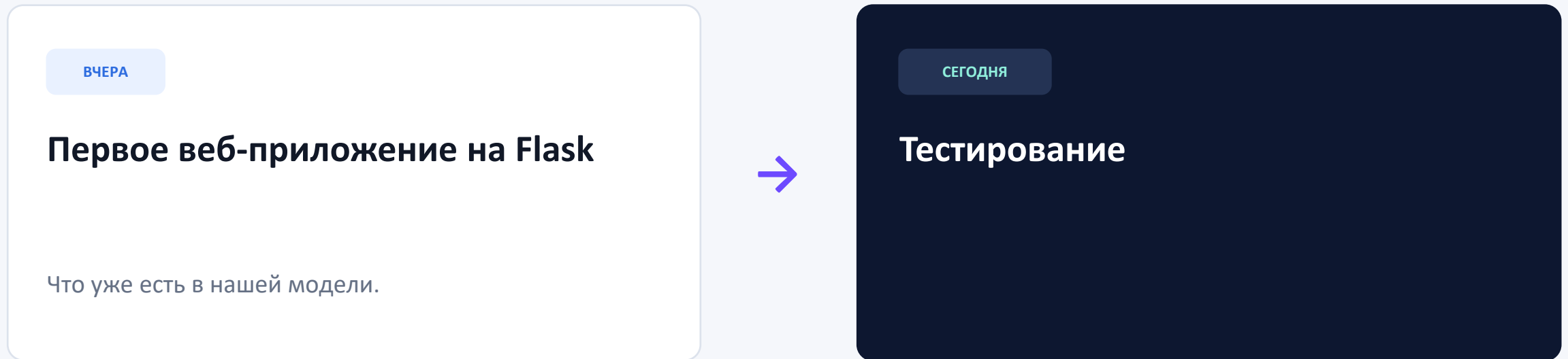
Конкретные действия вместо списка терминов.

- 01 Писать простые unit-тесты, понимать assert, fixtures/mocks на базовом уровне и различать unit, integration и regression testing.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Первое веб-приложение на Flask. Сегодня мы добавляем следующий слой: тестирование.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Тесты позволяют менять код с меньшим страхом: если поведение сломалось, мы узнаём об этом сразу, а не от пользователя.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

**Тест — автоматизированный чек-лист:
даём функции конкретный вход и
проверяем, совпал ли реальный
результат с ожидаемым.**

ПЕРЕВОД В IT

test case

один автоматический сценарий проверки поведения

unit test

изолированная проверка небольшой единицы кода

integration test

проверка совместной работы нескольких компонентов

Слой 1: test case • unit test • integration test

Сначала смысл, затем профессиональный термин.

**test case**

один автоматический сценарий проверки поведения

**unit test**

изолированная проверка небольшой единицы кода

**integration test**

проверка совместной работы нескольких компонентов

Слой 2: assert • mock • coverage

Сначала смысл, затем профессиональный термин.

**assert**

утверждение об ожидаемом результате

**mock**

контролируемая замена внешней зависимости в тесте

**coverage**

метрика кода, выполненного во время тестов

Слой 3: fixture • regression test

Сначала смысл, затем профессиональный термин.

fixture

подготовленные данные и окружение для теста

regression test

тест против повторного появления исправленной ошибки

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: test suite • edge case

Сначала смысл, затем профессиональный термин.

test suite

набор связанных автоматических тестов

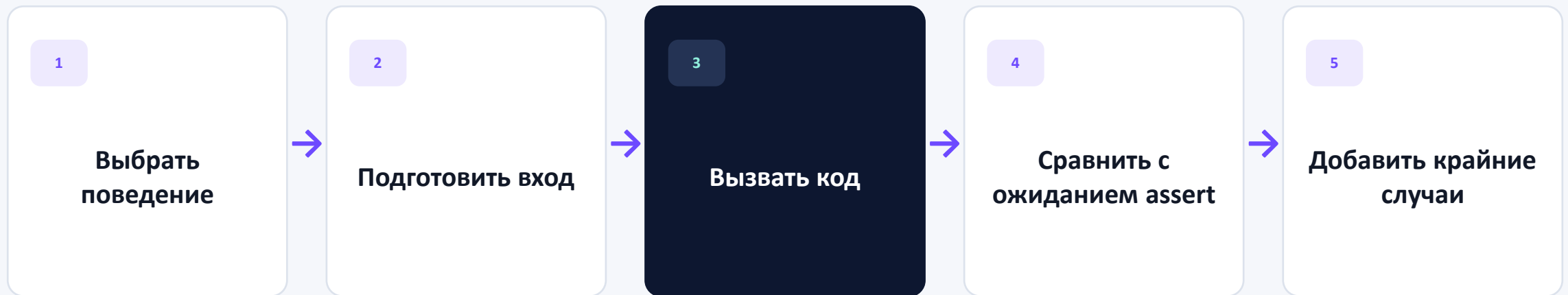
edge case

редкий пограничный сценарий работы программы

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day26_step1

```
def calculate_total(price, quantity):  
    return price * quantity  
  
assert calculate_total(150, 3) == 450
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
def calculate_total(price, quantity):
```

Создаём функцию и даём ей имя; код внутри выполнится при вызове.

```
    return price * quantity
```

Python выполняет эту строку сверху вниз.

```
assert calculate_total(150, 3) == 450
```

Сохраняем значение или результат вычисления под понятным именем.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
def calculate_total(price, quantity):  
    return price * quantity  
  
assert calculate_total(150, 3) == 450
```

РЕЗУЛЬТАТ

Тест проходит молча: 450 == 450

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
def is_adult(age):  
    return age >= 18  
  
assert is_adult(18) is True  
assert is_adult(17) is False
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Обе границы работают ожидаемо

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
def add(a, b):  
    return a + b  
  
assert add(2, 3) == 6
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Код функции корректен, а ожидание теста неверно. Хороший тест тоже нужно проверять: $2 + 3$ равно 5.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
def add(a, b):  
    return a + b  
  
assert add(2, 3) == 5
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Проверить функцию скидки на обычной цене и на скидке 0%.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Выбираем маленькую чистую функцию.**
- 2 Проверяем обычный сценарий.
- 3 Проверяем граничный сценарий 0%.
- 4 Используем точные ожидаемые значения.
- 5 Если тест падает — читаем отличие `expected/actual`.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
def discount(price, percent):  
    return price * (1 - percent / 100)
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.

```
def discount(price, percent):  
    return price * (1 - percent / 100)
```

```
assert discount(200, 10) == 180  
assert discount(200, 0) == 200
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Оба assert проходят

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Напиши тесты для `normalize_name(" anna ") -> "Anna"`.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Проверь пробелы, уже нормальное имя и пустую строку.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
def normalize_name(text):  
    return text.strip().title()  
  
assert normalize_name("  anna ") == "Anna"  
assert normalize_name("Max") == "Max"  
assert normalize_name("  ") == ""
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Три сценария проходят

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Спроектируй тесты endpoint `/square/5`: статус и содержимое JSON.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
# идея с Flask test client
response = client.get("/square/5")
assert response.status_code == 200
assert response.get_json()["square"] == 25
```

mini_project

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Проверены и HTTP-статус, и данные ответа

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Тестировать реализацию вместо поведения.

02

Писать только happy path.

03

Делать один тест на десятки несвязанных вещей.

04

Игнорировать падающие тесты как «ложную тревогу».

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «test case» и какую проблему это понятие помогает описать?

2

Чем «test case» отличается от «assert»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 test case — один автоматический сценарий проверки поведения.

2 test case: один автоматический сценарий проверки поведения. assert: утверждение об ожидаемом результате.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **Тест проходит молча: `450 == 450`**

4 **Код функции корректен, а ожидание теста неверно. Хороший тест тоже нужно проверять: `2 + 3` равно 5.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Писать простые unit-тесты, понимать `assert`, `fixtures/mocks` на базовом уровне и различать `unit`, `integration` и `regression testing`.
- ✓ Читать и объяснять пример темы «Тестирование».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Чистый код и code review

Завтра: Чистый код и code review. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.